AutoJudge: Predicting Programming Problem Difficulty

Rishabh Singhal
23113127

Introduction

Online competitive programming platforms such as Codeforces, CodeChef, and Kattis categorize programming problems based on difficulty levels (Easy, Medium, Hard) and often assign a numerical difficulty score. These difficulty labels are usually determined through a combination of expert judgment and user feedback, making the process subjective and time-consuming.

The objective of this project is to develop an automated system that predicts:

1. Problem Difficulty Class (Easy / Medium / Hard) — a classification task
2. Problem Difficulty Score — a regression task

The prediction is performed using only the textual content of the problem, including:

- Problem description
- Input description
- Output description

No metadata such as submissions, acceptance rates, or tags is used. The final system also includes a web-based interface that allows users to input a new problem description and obtain predicted difficulty outputs.

Dataset Description

The dataset used in this project consists of programming problems collected from online coding platforms. Each data sample contains the following attributes:

Column Name	Description
Title	Title of the programming problem
Description	Main problem description
input_description	Description of input format
output_description	Description of output format
sample_io	Sample input-output examples
problem_class	Difficulty class (Easy / Medium / Hard)
problem_score	Numerical difficulty score
url	Source URL of the problem

The dataset is pre-labeled and no manual labeling was performed as part of this project.

Data Preprocessing

Text Normalization

The textual fields may contain missing values, lists, or dictionaries. A normalization function was applied to:

- Convert lists and dictionaries into readable text
- Replace missing values with empty strings
- Convert all text into string format

Text Cleaning

The following cleaning steps were applied:

- Conversion to lowercase
- Removal of extra whitespace
- Concatenation of all text fields into a single feature

The final combined text field is defined as:

```
full_text = title + description + input_description + output_description + sample_io
```

Target Variable Processing

- Classification Target:
  Difficulty labels were mapped to integers:
  - Easy → 0
  - Medium → 1
  - Hard → 2
- Regression Target:
  The difficulty score was converted to numeric format and invalid entries were removed.

Feature Engineering

To capture both semantic and structural aspects of problem descriptions, multiple feature types were used.

TF-IDF Features

- Term Frequency–Inverse Document Frequency (TF-IDF) was used to represent textual content.
- Unigrams and bigrams were included.
- Stopwords were removed.
- Maximum number of features was limited to control dimensionality.

Handcrafted Features

Additional handcrafted features were extracted to capture problem complexity:

Feature	Description
Text length	Total number of characters
Math symbol count	Count of operators such as +, -, *, /, %, <, >
Keyword frequency	Occurrence of algorithmic keywords

Keywords used:

graph, tree, dp, dynamic programming, recursion, greedy, binary search, matrix, modulo, shortest path, dfs, bfs, segment tree

Final Feature Representation

TF-IDF features were combined with handcrafted features using a feature union approach to form the final feature matrix.

Model Design

Two independent machine learning tasks were modeled.

1. Classification Models

The following classifiers were trained and evaluated:

- Logistic Regression
- Support Vector Machine (Linear SVM)
- Random Forest Classifier

After comparison, Random Forest Classifier was selected as the final model based on overall performance.

2. Regression Models

The following regressors were trained:

- Linear Regression
- Random Forest Regressor
- Gradient Boosting Regressor

The Random Forest Regressor showed the most stable performance and was selected as the final regression model.

Experimental Setup

Dataset split: 80% training, 20% testing
Stratified sampling used for classification
Evaluation metrics:

- Classification: Accuracy, Confusion Matrix, Precision, Recall, F1-score
- Regression: MAE, RMSE

All experiments were conducted using Python and scikit-learn

Results and Evaluation

Classification Results

Overall Accuracy: ~52.9%

```
Accuracy: 0.528554070473876
              precision    recall  f1-score   support

        Easy       0.70      0.26      0.38       153
      Medium       0.40      0.20      0.27       281
        Hard       0.54      0.87      0.67       389

    accuracy                           0.53       823
   macro avg       0.55      0.44      0.44       823
weighted avg       0.52      0.53      0.48       823
```

Observations:

- The model performs best on Hard problems, achieving high recall.
- Easy and Medium problems are harder to distinguish due to overlapping textual patterns.
- Class imbalance influences predictions toward harder problems.

Regression Results

Mean Absolute Error (MAE): 1.60
Root Mean Squared Error (RMSE): 1.92

```
MAE: 1.6038736330498178
RMSE: 1.9219470600988469
```

Observations:

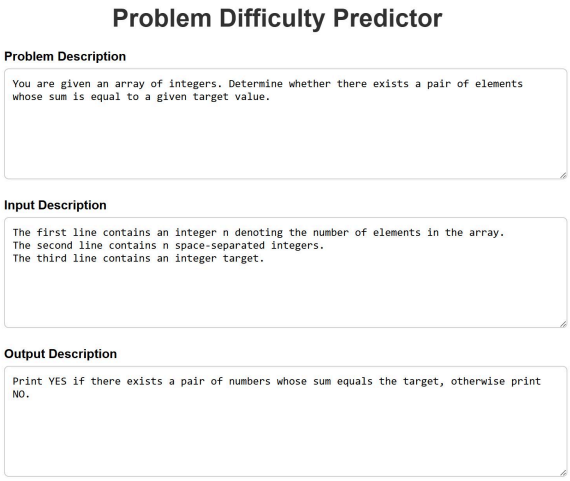
- The model captures general difficulty trends.
- Exact score prediction is challenging due to subjectivity in difficulty scoring.

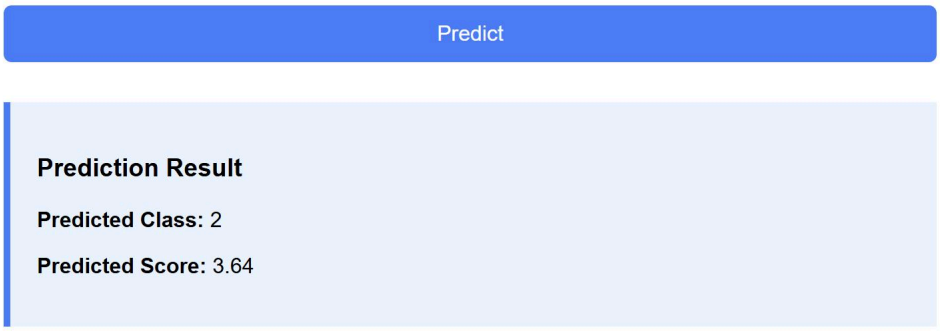
Web Interface

A simple web interface was developed using Flask and HTML.

Interface Features

- Text boxes for:
  - Problem description
  - Input description
  - Output description
- Predict button
- Display of:
  - Predicted difficulty class
  - Predicted difficulty score





Conclusion

This project demonstrates that programming problem difficulty can be reasonably predicted using only textual descriptions. While classification performance is moderate, the system effectively identifies harder problems and captures overall difficulty trends.
The results highlight the challenges posed by subjective difficulty definitions and overlapping textual patterns. Feature engineering significantly improves performance over basic text representations.